sd16. Thread-Safe DB Connection Pool (LLD)

> Bank #22 (◇ aggregator, 🎯 LLD staple; promoted with the distributed set). Sibling of sd03's lock discipline and sd12's lease thinking; HikariCP is the reference implementation. JD tie-in: every Java platform service owns a pool config; the "why not just grow the pool" defeater is a senior tell.

Index

1. Crux & why hard

- Naive: open a connection per request → 3-way handshake + auth per query, DB `max_connections` exhausted under load, no fairness, leaked connections kill the fleet quietly.
- Minimum primitives: **bounded lending** (semaphore = total permits), **idle reuse** (queue), **borrow timeout** (fail fast, don't pile up), **leak-proof return** (try-with-resources), **health of the lent thing** (validate/evict).
- 🗣️ "It's a bounded resource lender: a Semaphore bounds totals, a queue holds idle, and the API makes returning impossible to forget."

2. Clarifying questions (assumed answers)

- Max pool size? → 20 (DB-bound, see §7); borrow timeout 500 ms
- Behavior at exhaustion? → block up to timeout, then throw (never unbounded wait)
- Multi-DB / sharding? → one pool per target; a router above, not inside
- Fairness? → default unfair (throughput); note fair mode exists

3. FR / NFR

Req	Target	Enforced by
borrow()	p99 < 1 ms warm	idle queue poll

Req	Target	Enforced by
Bounded	never > maxSize conns	Semaphore permits
No leaks	leak surfaces in logs	AutoCloseable wrapper + leak detector
Dead conns not lent	validate-on-borrow (cheap)	`isValid` / `SELECT 1` if idle > 30 s
Exhaustion behavior	timeout + clear error	`tryAcquire(timeout)`

4. API

```
borrow(timeout) -> PooledConn   (throws PoolExhausted)
PooledConn.close()              -> returns to pool (never closes raw socket)
pool.shutdown(drain_timeout)
stats() -> {total, idle, in_use, waiters, created, evicted, leaks}
```

5. Core design (key code)

```
import queue, threading, time
from contextlib import contextmanager

class Pool:
    def __init__(self, factory, max_size=20, borrow_timeout=0.5):
        self.factory = factory
        self.idle = queue.LifoQueue()          # LIFO: warm conns stay warm
        self.permits = threading.BoundedSemaphore(max_size)
        self.borrow_timeout = borrow_timeout

    @contextmanager
    def borrow(self):
        if not self.permits.acquire(timeout=self.borrow_timeout):
            raise TimeoutError("pool exhausted")
        conn = None
        try:
            try:
                conn = self.idle.get_nowait()
                if conn.idle_since + 30 < time.time() and not conn.is_valid():
                    conn.close_raw()
                    conn = self.factory()      # replace dead idle conn
            except queue.Empty:
                conn = self.factory()          # under permit: safe to create
            yield conn
            conn.idle_since = time.time()
            self.idle.put(conn)                # healthy return
        except Exception:
            if conn:
                conn.close_raw()               # broken: destroy, don't return
            raise
        finally:
            self.permits.release()
# with pool.borrow() as c: ... — return is impossible to forget; permit bounds total
```

```
class Pool implements AutoCloseable {
    private final BlockingQueue<PooledConn> idle = new LinkedBlockingDeque<>();
    private final Semaphore permits;
    private final ConnFactory factory;
    private final long validateIfIdleMs = 30_000;

    Pool(ConnFactory f, int maxSize) { this.factory = f; this.permits = new Semaphore(maxSize); }

    PooledConn borrow(long timeoutMs) throws Exception {
        if (!permits.tryAcquire(timeoutMs, TimeUnit.MILLISECONDS))
            throw new PoolExhaustedException();
        try {
            PooledConn c = idle.poll();
            if (c != null && c.idleMs() > validateIfIdleMs && !c.raw().isValid(1)) {
                c.destroy();
                c = null;                                   // dead idle conn
            }
            return c != null ? c : factory.create(this);    // create under permit
        } catch (Exception e) {
            permits.release();                              // permit never leaks
            throw e;
        }
    }

    void release(PooledConn c, boolean broken) {            // called by c.close()
        try {
            if (broken) c.destroy();
            else idle.offer(c.touch());
        } finally {
            permits.release();
        }
    }
}
// PooledConn implements AutoCloseable -> try (var c = pool.borrow(500)) { ... }
// Semaphore = single source of truth for "how many exist"; queue is just idle storage
```

6. Validation, eviction, leak detection — the extras that score

- Validate on borrow only when idle > 30 s (`isValid(1)` / `SELECT 1`) — always-validate doubles round-trips
- Background reaper: evict idle > idleTimeout; retire any conn older than maxLifetime (LB/DNS failover + firewall state safety)
- Leak detector: on borrow, schedule "if not returned in N s, log the borrow stack trace" — HikariCP's `leakDetectionThreshold`
- Shutdown: stop lending, drain idle (close), wait bounded for in-use, then force

7. Sizing math

- 🗣️ Pool ≈ `cores × 2` on the DB side, not "requests in flight": each conn is a DB worker; 1000 app threads over a 20-conn pool is *correct*, the pool is the throttle
- Little's law sanity: needed ≈ qps × avg conn-hold seconds (100 qps × 50 ms = 5 conns busy)
- ❓ "Pool exhausted, grow it?" → check hold time first (slow query? leaked txn?); growing past DB capacity *reduces* throughput

8. Failure modes

- DB restart → all idle dead: validate-on-borrow replaces lazily; maxLifetime churns the rest
- Network partition mid-borrow → borrower gets exception → return-broken path destroys, permit released
- Thundering rebuild after DB recovery → cap concurrent creates (create-under-permit already does)
- Slow query hogging conns → borrow timeouts + hold-time metric point at the query, not the pool

9. Trade-offs & defeaters

- Defeater: "grow the pool under load" — DB max_connections is the ceiling; past ~2×cores throughput *drops* (context switching, lock contention)
- Defeater: "test-on-every-borrow" — latency tax; idle-threshold validation + maxLifetime covers it
- Defeater: "unbounded wait on exhaustion" — hides incidents as latency; timeout + error + alarm on waiters
- LIFO idle (warm, prepared-statement caches hot) vs FIFO (even aging): LIFO default

10. Pop-ups

- ❓ Why Semaphore + queue instead of one synchronized structure? → permit bounds *existence* (idle + in-use + being-created); queue only stores idle — separating the two invariants keeps each trivially correct
- ❓ Fair vs unfair? → unfair = better throughput; fair Semaphore for strict FIFO under sustained contention
- ❓ Prepared statements? → cache per-connection; another reason LIFO reuse wins
- **L5:** HikariCP's ConcurrentBag: thread-local stripes to avoid queue contention; name it as the next optimization

11. Memorization card

- Semaphore(maxSize) bounds totals; LIFO idle queue; create-under-permit; borrow timeout throws
- Return via AutoCloseable; broken → destroy + release; permit released in finally, always
- Validate if idle > 30 s; maxLifetime retire; leak detector logs borrow stack
- Size ≈ DB cores × 2; pool is the throttle, not the bottleneck